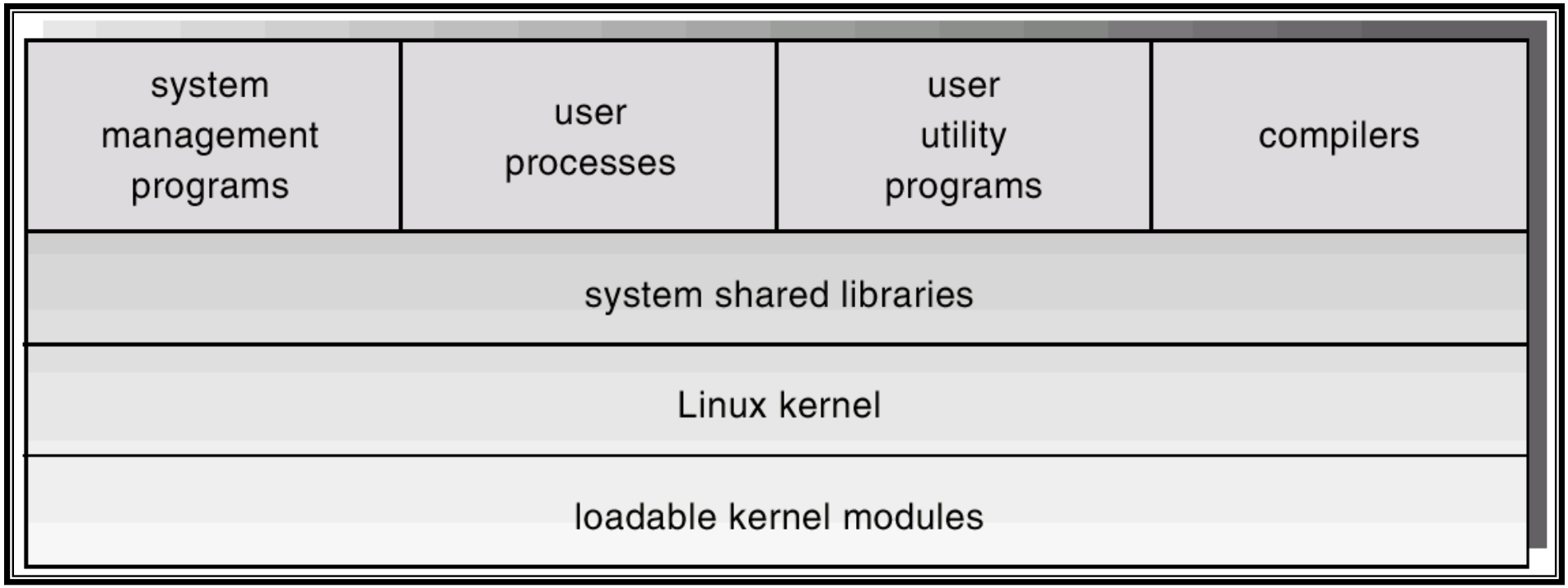
Overview

- Linux system
  - Kernel, system, distribution
  - Characteristics & goals
  - Kernel modules
- Process management
- CPU Scheduling, SMP, Interrupt handling
- Interprocess Communication
- Memory management
- File Systems
- Input and Output

Linux

- Kernel
  - Includes process management, memory management, device drivers, file systems, networking
  - Version 0.01 (first release): May 14, 1991
  - Textbook talks about Linux 2.6 kernel (late 2003)
  - Latest stable Linux kernel (http://www.kernel.org/): **3.7.6** 4/02/2013
- System
  - Includes various other components, such as network servers, web browsers, compilers, graphical user interface
- Distribution
  - Includes administrative tools to simplify installation and upgrading (e.g., Redhat, Ubuntu)
- Licensing under GNU General Public License (GPL): “free software”

Components of Linux



General Characteristics & Goals

- Multi-user, UNIX system
  - Full set of Unix tools
- Kernel
  - Monolithic-- single address space containing all kernel functionality (not a microkernel)
  - Modules
  - Multi-tasking
  - Preemptive: Processes can be preempted while running in kernel mode
- Goals
  - Speed, efficiency
    - E.g., install on a system with relatively small amount of RAM & disk
      - E.g., 4 MB RAM
  - Compatibility & standardization (e.g., POSIX)
    - “Even when the same system calls are present on two different UNIX systems, they do not necessarily behave in exactly the same way”

Kernel Modules

- Arbitrary sections or components of kernel code
- Run in kernel (privileged) mode
- Full access to hardware capabilities & instructions
- Modules can provide various capabilities
  - E.g., device driver, file system format, network protocol, binary excutable file format
- Kernel components (modules) can be dynamically loaded/unloaded
  - But still a single kernel address space

Explicit & Implicit Module Loading/Unloading

- Explicit
  - System can be configured so particular drivers are loaded upon system startup
- Implicit
  - Can be loaded on demand and unloaded when not in use
  - E.g., A CD-ROM driver might be loaded when a CD is mounted, and unloaded from memory when CD dismounted from file system
- Module support under Linux
  - *Module management*: Allows modules to be loaded into memory and to talk to rest of kernel
    - Module is dynamic linked into running kernel; kernel has a dynamic symbol table to allow module access to kernel symbols
  - *Driver registration*: Allows modules to tell rest of kernel that a new driver has become available
  - *Conflict-resolution mechanism*: Allows different device drivers to reserve hardware resources

Process Management

- Kernel support for both heavy-weight processes and threads
- PCB (process descriptor; p. 82, Linux Kernel Primer)
  - Process identity
    - PID, credentials (user & group ID), personality (emulation libraries)
  - Process environment
    - Command line arguments; shell variables
  - Process context (p. 750 text & p. 82, Linux Kernel Primer)
    - Scheduling context (e.g., registers), including kernel stack
    - Virtual memory context (region descriptions, page table)
    - Open file table
    - File system context
    - Signal handler table
    - Resource limits
- Same PCB structure for all process types
  - Thread shares some of data structures of parent
  - Each PCB is just a series of pointers into kernel tables

Process Management - 2

- System calls
  - Fork, exec, *clone*
- Linux doesn't distinguish between processes and threads
  - Uses the general term *task*
- When a clone is invoked, it is passed a set of flags that determine how much sharing is to take place between parent and child tasks (see Table, p. 750)
- Fork is just a special case of clone in which none of the process data is shared

CPU Scheduling

- Discussion is partly from 2.6.8.1 kernel CPU Scheduler (Aas, 2005)
- Preemptive, priority-based algorithm
- Multiple CPU process scheduling algorithms

  1) Time-sharing (“nice” scheduling)
     - Fair

       SCHED_NORMAL

  2) Real-time (RT)
     - Absolute priorities are more important than fairness

       SCHED_FIFO, SCHED_RR

Priorities & Time “Quanta”

- Numerically lower values indicate higher priorities
- Real-time priority values: 0-99
  - As long as there is a runnable real time (RT) task, no other tasks can run (Aas, 2005, p. 28)
- Time-sharing (“nice”) priority values: 100-140
- Variable time “quanta”
  - Higher priority processes (e.g., real-time) are given longer time quanta
  - E.g.,
    - SCHED_RR, priority 0 given 200 ms
    - Priority 140 given 10 ms

Linux Real-time Scheduling

- SCHED_FIFO, SCHED_RR
- Soft real-time scheduling
  - Priority-based, not deadline-based
- Each process has a priority & a scheduling class
  - Scheduling classes:
    - FCFS (first-come first-served)
    - RR (round-robin)
- Always runs highest priority process
  - Among equal priority processes, runs one that has been waiting the longest
- “FCFS” or “RR”
  - “The … difference between FCFS and round-robin scheduling is that FCFS processes continue to run until they either exit or block, whereas a round-robin process will be preempted after [a time slice] and will be moved to the end of the scheduling queue” (p. 753, text)
  - SCHED_FIFO (FCFS) processes do not have timeslices

Time-Sharing Scheduler

- SCHED_NORMAL
- Tasks initially given a time quanta according to priority
- Runnable tasks
  - Those that have time remaining in their quanta
- When a task exhausts its time quanta it has *expired*; otherwise it is *active*
  - Expired tasks are not scheduled again until all other tasks have exhausted their individual time quanta's
- Two arrays of tasks in the *runqueue*
  - Expired & active tasks
  - Each have linked lists within priority number
- When active array empty, swap with expired
- Tasks time quanta's replenished

SMP: Symmetric Multi-Processing

- Linux 2.0
  - Only one processor at a time could execute kernel code
  - Implemented with a single busy wait semaphore (spinlock, see p. 202 of text)
  - I.e., any other processes trying to gain access do a busy wait instead of blocking
- Linux 2.2
  - Multiple spinlocks in kernel
  - Limited execution of kernel code by more than one processor
- Linux 2.6
  - Generally, multiple tasks/processors can be using the kernel

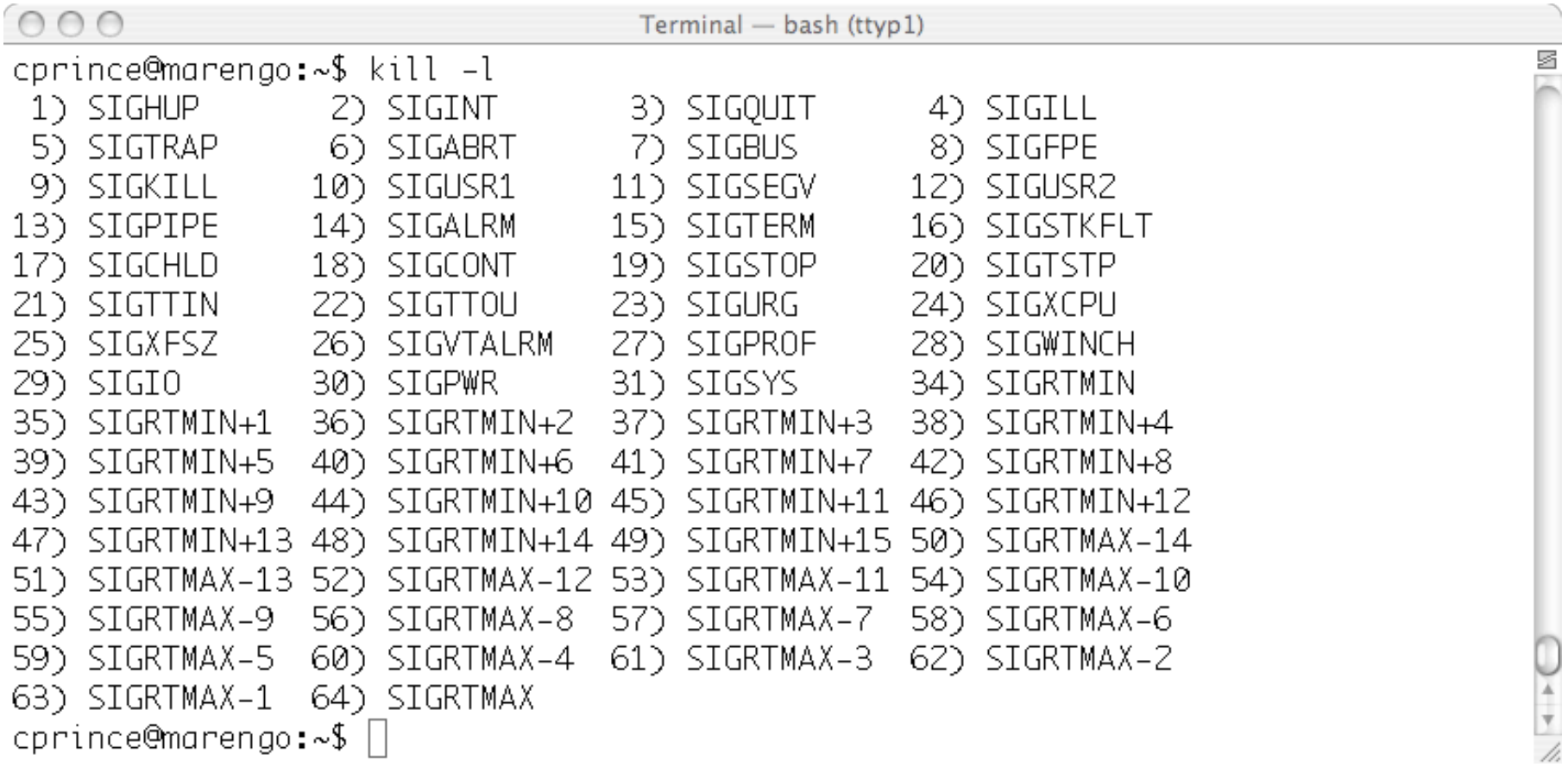
IPC: Interprocess Communication

- Semaphore
  - Can be used between heavy weight processes
- Pipe
  - Communication channel from parent to child
- Sockets
- Shared memory
  - Between light weight processes (in same HWP)
  - Between heavy weight processes
- Message queues
- Signals
  - Asynchronous events (not data); used to inform a process that an event has occurred
  - Sent by one user process to another or from kernel to a user process (e.g., to inform when a child dies)

Signals

- Some signals can be handled (caught) by the process
  - Handling takes the form of registering a signal handler
  - Signal handlers are functions that get called when the signal occurs
- Signal handling can be used to make a program more robust to certain conditions
  - E.g., if you want to make sure to do some processing before terminating a process, handling a signal generated by a ^C is a good idea
- Some signals cannot (e.g., SIGKILL, SIGSTOP) be handled by the process

Signals


Terminal — bash (ttyp1)
cprince@marengo:~$ kill -l
 1) SIGHUP   2) SIGINT   3) SIGQUIT   4) SIGILL
 5) SIGTRAP   6) SIGABRT   7) SIGBUS   8) SIGFPE
 9) SIGKILL   10) SIGUSR1   11) SIGSEGV   12) SIGUSR2
13) SIGPIPE   14) SIGALRM   15) SIGTERM   16) SIGSTKFLT
17) SIGCHLD   18) SIGCONT   19) SIGSTOP   20) SIGTSTP
21) SIGTTIN   22) SIGTTOU   23) SIGURG   24) SIGXCPU
25) SIGXFSZ   26) SIGVTALRM   27) SIGPROF   28) SIGWINCH
29) SIGIO   30) SIGPWR   31) SIGSYS   34) SIGRTMIN
35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3 38) SIGRTMIN+4
39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12
47) SIGRTMIN+13 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14
51) SIGRTMAX-13 52) SIGRTMAX-12 53) SIGRTMAX-11 54) SIGRTMAX-10
55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7 58) SIGRTMAX-6
59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
cprince@marengo:~$

Some Signal System Calls

int sigaction(int signum, const struct sigaction *act,
    struct sigaction *oldact);
    // Examine and change a signal action

sighandler_t signal(int signum, sighandler_t handler);
    // install a new signal handler (deprecated)

int pause(void); // wait for a signal

int kill(pid_t pid, int sig); // send a signal

unsigned int alarm(unsigned int seconds);
    // arranges for a SIGALRM signal to be delivered

Signal Example: Receiving Process

```
/* signal.c starts */

#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

void handleSIGINT(int sig)
{
        printf("received SIGINT (sig= %d)\n", sig);
        fflush(stdout);
}

void handleSIGQUIT(int sig)
{
        printf("received SIGQUIT (sig= %d)\n", sig);
        fflush(stdout);
}

void handleSIGUSR1(int sig)
{
        printf("received SIGUSR1 (sig= %d)\n", sig);
        fflush(stdout);
}

void handleSIGTSTP(int sig)
{
        printf("received SIGTSTP (sig= %d)\n", sig);
        fflush(stdout);
}

void handleSIGFPE(int sig)
{
        printf("received SIGFPE (sig= %d)\n", sig);
        fflush(stdout);
}
```

```
int main() {
    printf("pid of handler process is: %d    \n",
        getpid());
    signal(SIGINT, handleSIGINT);
    signal(SIGQUIT, handleSIGQUIT);
    signal(SIGTSTP, handleSIGTSTP);
    signal(SIGUSR1, handleSIGUSR1);
    signal(SIGFPE, handleSIGFPE);

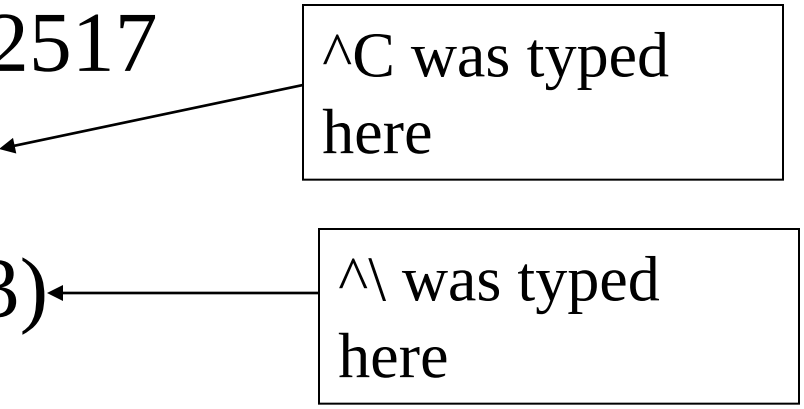
    for (;;) {
        printf("Waiting for signals...    \n");
        printf ("Enter 't' to test FPE exception; 'e' to exit  \n");
        int c = getchar();
        getchar(); // consume newline
        if (c == 'e') break;

        // No FPE is caused here
        float x = 100.0;
        float y = x/0.0;
        printf("y= %f  \n", y);

        //  FPE is caused here-- also causes an infinite loop
        int i=1/0;
    }
}
```

Signal Example: Signals from Keyboard

```
$ ./signal
pid of handler process is: 2517
received SIGINT (sig= 2)
received SIGQUIT (sig= 3)
```



Floaing Point Exceptions

- Seems that instruction pointer doesn't get advanced, so FPE happens repeatedly
  - http://technopark02.blogspot.com/2005_10_01_archive.html
- See also: http://ds9a.nl/fp/

Signal Example: Signals from another process - 1

```
#include <signal.h>
#include <stdio.h>

int main() {
    int pid;

    printf("Enter pid of process to send signal to: ");
    scanf("%d", &pid);

    kill(pid, SIGUSR1);
}
```

Signal Example: Signals from another process - 2

$ ./signal &

pid of handler process is: 2700

$ ./send

Enter pid of process to send signal to: 2700

received SIGUSR1 (sig= 10)

Linux Memory Management

- Internal kernel memory needs
  - Kmalloc kernel allocator
  - Slab allocator
  - Buddy heap
- Virtual memory

Kmalloc

- Memory allocator used by kernel routines (e.g., interrupt handlers)
  - E.g., they don't use "new Object" calls in C++
  - Need more explicit control over memory allocation
- Allocates variable amounts of memory
  - Analogous to malloc

    void *malloc(size_t size); // allocate size bytes of memory
- Acquires entire frames, and then splits into smaller pieces
- Allocates until explicitly freed; frames are locked and cannot be victimized

Slab Allocation

- Use to allocate kernel data structures
  - Slab: >= 1 physically contiguous frames
- Cache
  - Each cache: one or more slabs
  - Single cache for each kernel data structure
    - E.g., file objects, semaphores
  - Cache populated with objects (e.g., semaphore objects)

Buddy heap

- Used for allocation of contiguous regions
- A kind of “best fit” algorithm
- Emphasis on contiguous regions for reasons including
  - a) DMA requires contiguous frames (doesn’t use MMU)
  - b) Need contiguous pages for slabs
- Free frames are maintained on lists of 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 frames
- Release operation tries to iteratively merge double sized regions

Virtual Memory

- Pure demand paged virtual memory (p. 763)
- LFU-type page replacement policy
  - Form of clock algorithm
- Copy-on-write mechanisms (e.g., fork)
- Two separate views of a processes address space
  - Set of *regions*
  - Set of pages

Regions

- Logical address space: series of regions
  - No overlap between regions
  - Continuous sequence of pages of logical address space
  - E.g., regions for program text (code), data, stack
- Information per region (vm_area_struct)
  - Read, write, execute permissions for process
  - Any files associated with region
  - Table of function pointers for page management functions
  - Region type
- Region index structure (per process)
  - Allows lookups of region by logical address
  - Balanced binary tree

Region Types Defined By

- Backing store
  - "demand zero"
    - When process first tries to access (read or write) page in region, a frame is allocated, entered into page table, and it is initialized with zeros
  - File backing
    - Virtual-memory page is a *viewport* onto page of the file contents
- Reaction to writes
  - Private: copy-on-write
    - First write causes a new frame to be allocated, and contents copied prior to write
  - Shared: frame is updated

Page Tables

- Properties per page
  - Physical memory frame, or location on swap disk if swapped out
  - Resident or nonresident?
  - Read only?
  - Copy-on-write?
  - Age
- When page is accessed, but it is not resident:
  - Lookup the region for the virtual address
  - If needed (e.g., file backing region), the file is obtained for this page of the region
  - Region page-management functions are called
    - e.g., to bring the page in from disk
  - Physical frame is allocated
  - Page table updated

fork System Call Implementation

- Creates a new child process
  - Child receives copies of the parent region descriptors and page tables
- Reference count of frame for each resident page in parent is incremented
  - Parent and child now share same frames of memory
- Private regions: local writable data in parent
  - Both parent & child page table entries set to read-only
    - And marked for copy-on-write
  - If either process tries to modify a copy-on-write page
    - Reference count of frame checked
    - Page still shared? If yes, then:
      - Copy to new frame
      - Decrement reference count of source frame
      - Unmark copy-on-write of page in writing process
    - Set page to read/write in writing process

Swapping & Paging

- Linux does not implement whole process swapping
- Relies exclusively on paging
- Paging system divided into two sections
  - Policy algorithm
    - Which pages to write to disk
    - When to write to disk
  - Paging mechanism
    - Transfer of frame data to/from disk
- Policy algorithm: Pageout
  - Modified version of clock (or second-chance) algorithm
  - Age of page adjusted on each pass of clock
  - Age measures how much activity the page has seen recently
  - Pages selected as victims based on LFU policy

Linux File System

- Retains Unix broad concept of files– anything capable of handling the input or output of a stream of data
  - Regular files & directories on disk, device drivers, IPC, network connections
  - Also: Process, kernel, & device info (*proc* file system)
- Abstracts all of these kinds of data into stream input/output via *Virtual File System* (VFS)

VFS (Virtual File System)

- Objects
  - *inode object*: individual file
  - *file object*: current read/write point in open file
  - *superblock object*: represents an entire file system
  - *dentry object*: represents an individual directory entry
- For each type of object (e.g., a disk file or a network connection), a set of operations have to be defined
  - Every object contains a pointer to a function table that references the function the needed operations for the kind of object
  - For example, for disk files for file objects
    - int open(…) // open disk file
    - ssize_t read(…) // read from disk file
    - ssize_t write(…) // write to disk file
    - int mmap(…) // memory map a file

Linux Disk File System Formats: ext2fs

- "second extended file system"
- Tries to allocate logically adjacent blocks of a file into physically adjacent blocks on disk
  - In order to cluster physically adjacent I/O requests
  - To improve performance
- *Block Group*: a contiguous block sequence
  - Disk file system partitioned into multiple block groups
- Block allocation
  - Keep related information (disk blocks) in the same block group
    - File: attempts to keep in same block group as inode
    - Nondirectory inode: same block group as parent inode
    - Directory inode: dispersed to other block group
  - Bit map of free blocks in a block group

Input & Output

- General concept
  - All device drivers appear as files in the file system

    marengo(31): tty
    /dev/pts/3
    marengo(32): cat > /dev/pts/3
    hi there
    hi there
    marengo(33):
- Three device classes
  - Block devices: random access to fixed-size blocks of data
  - Character devices
  - Network devices

Interrupt Service Handlers

- In some cases, turning off interrupts is a poor solution
  - The interrupt handler is long running
  - Multi-processor
- Linux splits interrupt handlers into two parts
  - Top half
    - “normal” interrupt service routine
    - Prioritized interrupt structure
    - Only interrupts with higher priority can interrupt
  - Bottom half (for longer portion of interrupt handlers)
    - Runs with all interrupts enabled
    - Scheduled after running top half of handler
    - Synchronized kernel critical sections for running bottom halves
    - Bottom halves run using simple scheduler

Block Devices (e.g., hard disk, CD ROM)

- Block buffer cache
  - Pool of buffers for active and completed I/O
  - Frames (RAM) obtained from kernel's main memory pool
  - Each frame split into a number of equally sized buffers
- Each buffer can be held on a number of lists
  - Separate lists for clean, dirty, locked, and free buffers
- Data structure per buffer: *buffer_head*
  - Identity
    - device, offset within block device, size of buffer
  - lock, dirty time
- Hash lookup list for buffers not on free list

Block Devices: Request Manager

- Request manager
  - Software that manages the reading and writing of buffer contents to and from a block-device driver
- Request structure
  - list of buffer_head's describing I/O to be performed on a single device, in a contiguous range of sectors
- Separate list of requests for each block device driver
  - C-SCAN scheduling
  - Sorted order by increasing starting-sector number
  - Attempts to merge requests in per-device lists

Bypassing Buffer Cache

- Some subsystems handle I/O somewhat differently
  - e.g., virtual memory system accessing swap device
  - Still go through request manager
  - Create temporary buffer_head's for purpose of submitting I/O requests to request manager
  - Once the I/O is complete the buffer_head is discarded
  - Use buffer_head's to label a page of memory for active I/O only
  - E.g., a buffer_head to describe a request that a sector from the swap device be read into a particular frame of RAM to handle a page fault

Character & Network Devices

- Character devices
  - Devices that don't provide random access to blocks of data
    - Character-device device driver registers a set of functions that implement byte by byte I/O
    - I/O operation just passed to device driver
  - Terminal devices are handled specially; make use of line disciplines; e.g., tty discipline
    - Buffering & flow control on the data stream for terminal device
    - Manages connecting standard input/output streams to running processes
      - Different processes can be obtaining input/output to the terminal over time
- Network devices: Data transferred through networking subsystem of kernel